PROJET PYTHON · PRÉSENTATION

# Le jeu de Nim

Un jeu de stratégie à deux joueurs, en trois fichiers.

**Mathis** · main.py

**Marine** · nim_logique.py

**Jauris** · nim_affichage.py



LES RÈGLES

# Retirer le dernier objet pour gagner

1. Les objets sont répartis en **plusieurs tas** (en configuration classique : 3, 5 et 7).
2. À son tour, un joueur retire **au moins 1 objet**, dans **un seul tas** de son choix.
3. Les deux joueurs jouent **chacun leur tour**.
4. Celui qui retire **le dernier objet gagne** la partie.

```
Etat des tas :
Tas 0 : |||
Tas 1 : |||||
Tas 2 : |||||||
```

ORGANISATION

# Un projet en trois fichiers

**Mathis**

## `main.py`

Lance et organise la partie : menu, tours de jeu, fin de partie.

**Marine**

## `nim_logique.py`

Contient les règles : tas, validité d'un coup, fin de partie et conseil.

**Jauris**

## `nim_affichage.py`

Gère ce que les joueurs voient et ce qu'ils saisissent.

**main.py** importe les deux autres fichiers. **nim_affichage.py** réutilise aussi **coup_valide** et **conseiller_coup** de nim_logique.py.

Mathis `main.py`

# Le programme principal : il lance et organise la partie

1
### Afficher les règles
Dès le lancement du programme.

2
### Choisir le mode
Classique, tas personnalisés ou quitter.

3
### Initialiser les tas
Le joueur 1 commence.

4
### Jouer chacun son tour
Demander un coup, l'appliquer, actualiser l'affichage.

5
### Annoncer le gagnant
Quand tous les tas sont vides.

6
### Proposer de rejouer
Sinon, le programme se termine.

Mathis main.py · jeu_nim()

# La boucle de jeu

```
while not partie_terminee(tas):
    numero_tas, nombre_objets = demander_coup(tas, numero_joueur)
    appliquer_coup(tas, numero_tas, nombre_objets)
    effacer_console()
    afficher_regles()
    afficher_tas(tas)

    if partie_terminee(tas):
        annoncer_gagnant(numero_joueur)
    else:
        numero_joueur = 3 - numero_joueur
```

### Boucle de partie

Répétée tant qu'il reste des objets.

### Un tour de jeu

Coup demandé, appliqué, puis écran actualisé.

### Joueur suivant

3 − numero_joueur alterne entre 1 et 2.

Extrait de main.py

Marine `nim_logique.py`

# Les règles et les opérations du jeu

### `initialiser_tas`

Crée les tas. Par défaut : 3, 5 et 7.

### `coup_valide`

Vérifie qu'un coup respecte les règles.

### `appliquer_coup`

Retire les objets du tas choisi.

### `partie_terminee`

Vrai quand tous les tas sont vides.

### `conseiller_coup`

Propose un coup grâce au XOR.

### Logique pure

Ce fichier ne lit rien et n'affiche rien.

Marine `nim_logique.py · coup_valide`

# Vérifier qu’un coup est valide

```
def coup_valide(tas, numero_tas, nombre_objets):
    if numero_tas < 0 or numero_tas ⩾ len(tas):
        return False
    if nombre_objets < 1:
        return False
    if nombre_objets > tas[numero_tas]:
        return False
    return True
```

### Le tas existe

Son numéro est dans la liste des tas.

### Au moins 1 objet

Retirer 0 objet est refusé.

### Pas plus que le tas

On ne retire pas plus d’objets qu’il n’en reste.

Extrait de nim_logique.py (commentaires retirés)

Marine nim_logique.py · conseiller_coup

# Conseiller un coup

```python
def conseiller_coup(tas):
    somme_nim = 0
    for nombre_objets in tas:
        somme_nim ^= nombre_objets

    if somme_nim == 0:
        return None

    for numero_tas, nombre_objets in enumerate(tas):
        cible = nombre_objets ^ somme_nim
        if cible < nombre_objets:
            return numero_tas, nombre_objets - cible

    return None
```

### Somme de Nim

XOR de tous les tas (l'opérateur ^).

### Somme égale à 0

Aucun coup garanti : renvoie None.

### Somme non nulle

On cherche un tas à réduire jusqu'à la valeur cible.

Extrait de nim_logique.py (commentaires retirés)

Jauris `nim_affichage.py`

# Ce que les joueurs voient et saisissent

## Ce que les joueurs voient

**`afficher_regles`**
Règles du jeu et rappel du conseil « c »

**`afficher_tas`**
Une barre par objet restant dans chaque tas

**`annoncer_gagnant`**
Message de victoire du dernier joueur

**`effacer_console`**
cls ou clear, selon le système d'exploitation

## Ce que les joueurs saisissent

**`demander_mode_jeu`**
Menu : classique, personnalisé ou quitter

**`demander_configuration`**
Nombre de tas et d'objets dans chacun

**`demander_coup`**
Tas, objets retirés, ou « c » pour un conseil

**`demander_entier`**
Recommence tant que ce n'est pas un entier

**`demander_rejouer`**
Réponse « o » ou « n »

Jauris nim_affichage.py · afficher_tas

# Représenter les tas avec des barres

```python
def afficher_tas(tas):
    print("\nEtat des tas :")
    for indice, nombre_objets in enumerate(tas):
        symboles = "|" * nombre_objets
        print(f"  Tas {indice} : {symboles}")
    print()
```

```
Etat des tas :
Tas 0 : |||
Tas 1 : |||||
Tas 2 : |||||||
```

**Une barre par objet**

La chaîne de barres a la longueur du tas.

Extrait de nim_affichage.py · résultat avec la configuration classique

CONCLUSION

# Projet dispo sur github

https://github.com/Bananoob/Nimpy

